

UNIwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych



Piotr Prezydent
131497

Informatyka

*Projekt i implementacja rozproszonego asystenta AI z modułową
architekturą wtyczek*

Praca inżynierska

Praca wykonana pod kierunkiem
dr inż. Przemysław Pardel

Rzeszów 2026

Spis treści

1. Wstęp i cel pracy	7
1.1. Motywacja do podjęcia tematu	7
1.2. Cel pracy i oczekiwane rezultaty projektu	7
1.3. Problem badawczy i wyzwania techniczne	8
1.3.1. Kwestia zamkniętych ekosystemów i zjawisko <i>vendor lock-in</i>	8
1.3.2. Zarządzanie rozproszonym stanem i komunikacją w czasie rzeczywistym	8
1.3.3. Abstrakcja integracji z dużymi modelami językowymi i mechanizm <i>function calling</i>	8
1.4. Zakres pracy	8
1.4.1. Elementy systemu: serwer, klient oraz wtyczki	8
1.4.2. Ograniczenia i wykluczenia projektowe	9
1.5. Struktura pracy	9
2. Analiza dziedziny i przegląd technologii	10
2.1. Analiza dziedziny: Asystenci sztucznej inteligencji i systemy rozproszone	10
2.2. Przegląd technologii: Uzasadnienie stosu technologicznego	10
2.2.1. Platforma .NET i język C#	10
2.2.2. Architektura wtyczek jako biblioteki dynamiczne (.dll)	11
2.2.3. Komunikacja w czasie rzeczywistym i biblioteka SignalR	11
2.2.4. Abstrakcja modeli LLM za pomocą Semantic Kernel	11
2.2.5. Wzorce architektoniczne i bazy danych	11
3. Analiza i projektowanie systemu	12
3.1. Wymagania systemowe	12
3.1.1. Wymagania funkcjonalne	12
3.1.2. Wymagania нефункционалне	12
3.2. Modelowanie przypadków użycia systemu	13
3.2.1. Identyfikacja aktorów i głównych procesów biznesowych	13
3.2.2. Diagramy przypadków użycia	13
3.3. Architektura systemu	14
3.3.1. Architektura Modular Monolith i podział na konteksty (ang. <i>Bounded Contexts</i>)	14
3.3.2. Struktura warstwowa modułów (DDD i CQRS)	15
3.3.3. Architektura wtyczek na urządzeniach klienckich	15
3.4. Projekt bazy danych	15
3.4.1. Modelowanie danych z perspektywy domenowej	15
3.4.2. Diagram związków encji (ERD)	15
4. Implementacja systemu	17
4.1. Implementacja logiki serwera i komunikacji między modułami	17
4.1.1. Struktura i odpowiedzialność głównych modułów biznesowych	17

4.1.2.	Propagacja zdarzeń integracyjnych i wzorzec <i>Event-Driven</i>	17
4.2.	Implementacja po stronie klienta.....	17
4.2.1.	Mechanizm dynamicznego ładowania zestawów przy użyciu refleksji.....	17
4.2.2.	Rejestracja metadanych wtyczek na serwerze	17
4.3.	Implementacja przepływu przetwarzania zapytań użytkownika	17
4.3.1.	Agregacja metadanych i budowa kontekstu zapytania.....	17
4.3.2.	Wykorzystanie Semantic Kernel do parsowania poleceń	17
4.4.	Obsługa dwukierunkowej komunikacji przy użyciu SignalR	17
5.	Testowanie i walidacja	18
5.1.	Założenia i środowisko testowe	18
5.2.	Scenariusze testowe	18
5.2.1.	Testowanie procesów tworzenia i łączenia urządzeń w grupy.....	18
5.2.2.	Walidacja poprawności dynamicznego ładowania wtyczek	18
5.3.	Weryfikacja działania mechanizmu <i>function calling</i>	18
5.4.	Analiza wydajności rozgłaszania wiadomości za pomocą SignalR	18
6.	Podsumowanie	19
6.1.	Ocena stopnia realizacji celu pracy i wymagań.....	19
6.2.	Napotkane problemy i sposoby ich rozwiązania	19
6.3.	Perspektywy dalszego rozwoju aplikacji.....	19
	Bibliografia	20
	Spis rysunków	21
	Spis tabel	22
	Spis listingów	23
	Streszczenie pracy	24
	Abstract	26
	Oświadczenie studenta o samodzielności pracy	28
	Oświadczenie studenta o zgodności wersji papierowej i elektronicznej pracy	29

1. Wstęp i cel pracy

1.1. Motywacja do podjęcia tematu

W dobie powszechnej cyfryzacji i rozwoju sztucznej inteligencji (AI), asystenci głosowi i tekstowi, tacy jak Google Assistant, Amazon Alexa czy Apple Siri, stali się integralną częścią codziennego życia. Umożliwiają oni interakcję z inteligentnymi urządzeniami za pomocą języka naturalnego, co znacząco podnosi komfort i efektywność pracy. Niemniej jednak, większość z dostępnych na rynku rozwiązań komercyjnych cechuje się zamkniętością ekosystemu. Wprowadzenie obsługi nowego sprzętu lub rozszerzenie systemu o niestandardowe funkcje często wymaga integracji z dedykowanymi środowiskami programistycznymi (SDK) dostawców, przechodzenia przez skomplikowane procesy certyfikacyjne, a w niektórych przypadkach jest wręcz niemożliwe bez dostępu do kodu źródłowego rdzenia systemu.

Kolejnym istotnym problemem jest brak łatwej interoperacyjności pomiędzy urządzeniami pochodzącymi od różnych producentów. Zamknięte rozwiązania rzadko oferują spójny sposób komunikacji międzyplatformowej, co prowadzi do zjawiska uzależnienia od jednego dostawcy (ang. *vendor lock-in*). Dodatkowo, komercyjne asystenty opierają swoje działanie na chmurach obliczeniowych firm trzecich, co budzi uzasadnione obawy dotyczące prywatności użytkowników oraz bezpieczeństwa danych. Zauważalny jest również brak wbudowanych mechanizmów umożliwiających swobodne grupowanie posiadanych urządzeń (takich jak komputer osobisty, smartfon i smartwatch) we wspólną przestrzeń roboczą, którą można by zarządzać za pomocą jednego scentralizowanego zapytania, z dowolnego urządzenia należącego do grupy. Powyższe ograniczenia stały się główną motywacją do podjęcia prac nad otwartym, rozszerzalnym i zdecentralizowanym pod względem wykonawczym systemem asystenta AI.

1.2. Cel pracy i oczekiwane rezultaty projektu

Głównym celem niniejszej pracy inżynierskiej jest zaprojektowanie oraz implementacja rozproszonego, modularnego systemu asystenta AI, który umożliwi sterowanie wieloma różnorodnymi urządzeniami przy użyciu poleceń w języku naturalnym (tekstowych lub głosowych). System ma w założeniu stanowić otwartą platformę, w której dodawanie nowych urządzeń oraz wprowadzanie nowych funkcjonalności będzie odbywać się w sposób bezinwazyjny dla głównego jądra aplikacji, dzięki zastosowaniu architektury opartej na wtyczkach (ang. *plugins*).

W ramach realizacji projektu oczekiwane jest stworzenie w pełni funkcjonalnego prototypu, na który złożą się dwa główne komponenty. Pierwszym z nich jest aplikacja kliencka instalowana na urządzeniach końcowych, posiadająca zdolność dynamicznego ładowania zewnętrznych modułów wykonywalnych. Drugim elementem jest serwer centralny oparty na architekturze modularnego monolitu (ang. *Modular Monolith*), wykorzystujący podejście Domain-Driven Design (DDD). Serwer ma za zadanie zarządzać połączeniami, autoryzacją, grupowaniem urządzeń oraz koordynować przetwarzanie zapytań użytkownika przy wykorzystaniu dużych modeli językowych (LLM). Oczekiwany rezultatem jest również udowodnienie słuszności przyjętej architektury poprzez implementację zestawu przykładowych wtyczek (np. integracja z systemem plików, czy sterowanie wybranymi aplikacjami), które potwierdzą elastyczność i rozszerzalność opracowanego rozwiązania.

1.3. Problem badawczy i wyzwania techniczne

Realizacja tak złożonego systemu rozproszonego wymagała rozwiązania szeregu problemów badawczych i technicznych, obejmujących zarówno architekturę oprogramowania, jak i integrację nowoczesnych technik sztucznej inteligencji z klasycznymi mechanizmami komunikacji sieciowej.

1.3.1. Kwestia zamkniętych ekosystemów i zjawisko *vendor lock-in*

Kluczowym wyzwaniem badawczym było opracowanie mechanizmu pozwalającego na ominięcie ograniczeń narzucanych przez zamknięte ekosystemy producentów oprogramowania. Problem ten rozwiązano poprzez zaprojektowanie architektury wtyczek działających po stronie klienta. Wykorzystano tu właściwości platformy .NET, pozwalające na dynamiczne ładowanie skompilowanych bibliotek (plików .dll) w czasie działania programu. Mechanizm ten gwarantuje, że kod wykonawczy wtyczki pozostaje na urządzeniu końcowym, a serwer dysponuje wyłącznie jej metadanymi, co znacząco podnosi bezpieczeństwo i uniezależnia architekturę centralną od specyficznych implementacji dostawców sprzętu.

1.3.2. Zarządzanie rozproszonym stanem i komunikacją w czasie rzeczywistym

System, z uwagi na swoją specyfikę, wymaga utrzymywania stabilnej, dwukierunkowej komunikacji między węzłem centralnym (serwerem) a wieloma klientami, zminimalizowania opóźnień oraz obsługi dynamicznych zmian stanu (np. odłączenie urządzenia, zmiana konfiguracji wtyczek). Wyzwaniem technicznym w tym obszarze było zastosowanie odpowiedniego protokołu czasu rzeczywistego. Rozwiązanie to musiało obsługiwać nie tylko transport danych, ale także umożliwić tworzenie logicznych grup urządzeń oraz bezpieczne przekazywanie wywołań zdalnych (RPC) z serwera do konkretnych instancji klientów, a następnie odbiór wyników ich wykonania.

1.3.3. Abstrakcja integracji z dużymi modelami językowymi i mechanizm *function calling*

Współczesne duże modele językowe różnią się interfejsami API oraz formatami obsługi wywoływania funkcji (ang. *function calling*). Budowa systemu, który potrafi przełożyć intencję wyrażoną w języku naturalnym na konkretne wywołanie metody w aplikacji klienckiej, stanowi istotny problem technologiczny. Wyzwaniem było stworzenie mechanizmu, który w czasie rzeczywistym agreguje definicje funkcji ze wszystkich dostępnych urządzeń w grupie, buduje dynamiczny prompt rozszerzony o te funkcje, a następnie przetwarza odpowiedź modelu na ustrukturyzowane polecenia niezależnie od konkretnego dostawcy technologii LLM.

1.4. Zakres pracy

Z uwagi na szeroki kontekst dziedziny, w której osadzony jest projekt, konieczne było precyzyjne określenie jego zakresu, by dostarczyć spójne i mierzalne rezultaty w ramach pracy inżynierskiej.

1.4.1. Elementy systemu: serwer, klient oraz wtyczki

W zakres realizacji projektu wchodzi zaprojektowanie od podstaw architektury systemu. Po stronie serwerowej implementacja obejmuje stworzenie aplikacji biznesowej z podziałem na moduły odpowiadające za połączenia, grupy, wtyczki, zarządzanie promptami oraz wykonywanie zadań. Po stronie klienckiej zakres obejmuje implementację programu ładującego i zarządzającego lokalnymi modułami (wtyczkami)

75 oraz komunikującego się z serwerem. Ponadto, przygotowany zostanie zestaw podstawowych wytyczek de-
76 monstrujących działanie systemu, takich jak moduł zarządzania systemem plików czy monitorowania wy-
77 branych parametrów systemu operacyjnego.

78 1.4.2. Ograniczenia i wykluczenia projektowe

79 Projekt nie obejmuje tworzenia własnych, zaawansowanych algorytmów zamiany mowy na tekst (ang.
80 *Speech-to-Text*), a ewentualna interakcja głosowa realizowana będzie z wykorzystaniem gotowych, ze-
81 wnętrznych interfejsów API dostarczanych przez system operacyjny. Ponadto, serwer centralny nie został
82 rozproszony na mikroserwisy. Mimo iż architektura modułowego monolitu narzuca ściśle granice i poten-
83 cjalnie pozwala na taki podział w przyszłości, implementacja mikroservisów wprowadziłaby na tym etapie
84 nadmiarową złożoność operacyjną. Praca nie skupia się również na tworzeniu gotowego, bogatego eko-
85 systemu wytyczek gotowych do dystrybucji komercyjnej, lecz na opracowaniu niezawodnego fundamentu
86 umożliwiającego ich bezproblemowe tworzenie i wdrażanie w przyszłości.

87 1.5. Struktura pracy

88 Rozdział pierwszy pracy definiuje kontekst, cele i zakres projektu oraz omawia problemy badawcze.
89 W rozdziale drugim przedstawiono przegląd dostępnych na rynku rozwiązań w dziedzinie asystentów AI
90 oraz omówiono użyte w projekcie technologie i wzorce projektowe. Rozdział trzeci poświęcony jest szcze-
91 gółowemu opisowi architektury stworzonego systemu, z naciskiem na architekturę klient-serwer, podejście
92 Modular Monolith oraz Domain-Driven Design. Rozdział czwarty obejmuje analizę implementacji mecha-
93 nizmu wytyczek oraz integracji z modelami LLM przy użyciu Semantic Kernel. Rozdział piąty prezentuje
94 wyniki testów i weryfikację założeń projektowych na podstawie działania prototypu. Pracę kończy rozdział
95 szósty, zawierający podsumowanie osiągniętych rezultatów oraz wskazujący kierunki dalszego rozwoju
96 systemu.

2. Analiza dziedziny i przegląd technologii

2.1. Analiza dziedziny: Asystenci sztucznej inteligencji i systemy rozproszone

Rozwój interfejsów konwersacyjnych oraz dużych modeli językowych (LLM, ang. *Large Language Models*) zrewolucjonizował sposób interakcji człowieka z systemami komputerowymi. Współcześni asystenci sztucznej inteligencji przeszli ewolucję od prostych systemów opartych na predefiniowanych regułach i sztywnych drzewach dialogowych do zaawansowanych agentów zdolnych do rozumienia niuansów języka naturalnego. Prawdziwym przełomem, z punktu widzenia systemów sterowania, stało się wprowadzenie do modeli LLM mechanizmu wywoływania narzędzi (ang. *function calling*). Pozwala on modelowi nie tylko odpowiadać tekstem, ale również analizować dostarczone metadane funkcji, decydować, która z nich jest potrzebna do zrealizowania intencji użytkownika, i zwracać sformatowany zbiór argumentów (najczęściej w formacie JSON) gotowy do uruchomienia na urządzeniu.

Mimo tak znaczącego postępu w rozwoju algorytmów, praktyczne zastosowanie asystentów do zarządzania heterogenicznym środowiskiem urządzeń (komputery, smartfony, inteligentne zegarki) natrafia na poważne bariery rynkowe i technologiczne. Dominujące na rynku rozwiązania (takie jak Apple Siri, Amazon Alexa czy Google Assistant) rozwijane są w paradygmacie zamkniętych ekosystemów. Producenci wymuszają korzystanie z ich własnych środowisk programistycznych (SDK), zewnętrznej infrastruktury chmurowej oraz procesów certyfikacji, co skutecznie uniemożliwia swobodną rozbudowę systemu przez użytkownika końcowego. Zjawisko to, określane jako uzależnienie od dostawcy (ang. *vendor lock-in*), stanowi barierę dla interoperacyjności.

Zaprojektowana w ramach niniejszej pracy aplikacja stanowi odpowiedź na te problemy. Głównym celem jest dostarczenie rozproszonego, otwartego środowiska, w którym użytkownik może dynamicznie grupować swoje urządzenia, tworząc wspólną przestrzeń sterowania, a funkcjonalność każdego z nich może być dowolnie rozszerzana za pomocą niezależnych modułów (wtyczek), bez modyfikacji centralnego kodu serwera.

2.2. Przegląd technologii: Uzasadnienie stosu technologicznego

Wybór odpowiednich technologii, frameworków oraz narzędzi architektonicznych miał kluczowe znaczenie dla sprostania postawionym wymaganiom projektowym, w szczególności w zakresie wydajności, łatwości rozbudowy i stabilności komunikacji.

2.2.1. Platforma .NET i język C#

Jako główny język programowania dla całego systemu (zarówno serwera, jak i aplikacji klienckiej) wybrano język C# w oparciu o platformę .NET. Wybór ten podyktowany był przede wszystkim wieloplatformowością ekosystemu (obsługa systemów Windows, Linux, macOS oraz urządzeń mobilnych). Ścisłe typowanie języka C# znacząco ułatwia utrzymanie skomplikowanej domeny biznesowej (zgodnie z zasadami Domain-Driven Design). Ponadto, środowisko .NET dostarcza niezwykle rozwinięte mechanizmy refleksji (ang. *reflection*), które stanowią technologiczny fundament dla systemu wtyczek tworzonego po stronie klienta, umożliwiając dynamiczne analizowanie ładowanych komponentów i ekstrakcję ich metadanych w czasie działania aplikacji.

2.2.2. Architektura wtyczek jako biblioteki dynamiczne (.dll)

Dla systemu wtyczek (ang. *plugins*) uruchamianych na urządzeniach klienckich, zdecydowano się na wykorzystanie skompilowanych bibliotek współdzielonych w formacie `.dll`. Format ten jest natywny dla platformy .NET. Taki wybór eliminuje konieczność instalacji dodatkowych środowisk uruchomieniowych (np. silników interpretera dla skryptów Python lub JavaScript) po stronie klienta. Umożliwia to proces instalacji wtyczki sprowadzający się do prostego skopiowania pliku, oferując przy tym wysoką wydajność skompilowanego kodu oraz bezpieczeństwo (dzięki izolacji pamięciowej tzw. *AssemblyLoadContext*).

2.2.3. Komunikacja w czasie rzeczywistym i biblioteka SignalR

System sterowania urządzeniami wymaga dwukierunkowej komunikacji sieciowej charakteryzującej się niskimi opóźnieniami. Standardowe, bezstanowe podejście oparte na zapytaniach HTTP (ang. *Request-Response*) jest niewystarczające, ponieważ nie pozwala serwerowi na aktywne wysłanie polecenia do klienta, zmuszając go do ciągłego, nieefektywnego odpytywania serwera (ang. *polling*). Z tego względu zdecydowano się na wykorzystanie biblioteki SignalR. Zapewnia ona warstwę abstrakcji nad protokołem WebSockets (automatycznie obniżając warstwę transportową do Server-Sent Events lub Long Polling, gdy WebSockets nie są dostępne). Co niezwykle istotne w kontekście założeń pracy, SignalR posiada wbudowany mechanizm logicznych *Grup* połączeń, co idealnie pokrywa się z wymaganiem domenowym dotyczącym łączenia urządzeń we wspólne przestrzenie zarządzania.

2.2.4. Abstrakcja modeli LLM za pomocą Semantic Kernel

Integracja mechanizmu *function calling* w systemie wymaga transformacji metadanych lokalnych wtyczek na format zrozumiały dla dużego modelu językowego (LLM). Różni dostawcy modeli (np. OpenAI, Anthropic, Google) stosują zróżnicowane schematy zapytań API. W celu uniknięcia sprzęgnięcia logiki biznesowej z pojedynczym dostawcą technologii, zastosowano otwartą bibliotekę Microsoft Semantic Kernel. Działa ona jako zaawansowana warstwa pośrednicząca (ang. *middleware*), standaryzująca komunikację z modelami i zarządzająca formatowaniem zapytań (tzw. *prompt engineering*). Umożliwia to elastyczną zmianę używanego modelu bez konieczności przepisywania mechanizmów analizowania intencji.

2.2.5. Wzorce architektoniczne i bazy danych

Projekt centralnego serwera zrealizowano w architekturze modułowego monolitu (ang. *Modular Monolith*) przy użyciu metodyki Domain-Driven Design (DDD). Decyzja o rezygnacji z popularnej architektury mikroservisowej podyktowana była chęcią uniknięcia nadmiarowego narzutu operacyjnego związanego z wdrażaniem i monitorowaniem wielu rozproszonych komponentów. Modułowy monolit dostarcza wyraźne granice modułów (ang. *Bounded Contexts*) – takie jak połączenia, grupy czy przetwarzanie promptów – oferując podobną jakość separacji logiki biznesowej co mikroservisy, jednak w obrębie pojedynczego procesu. Do persystencji danych wybrano relacyjną bazę danych (np. PostgreSQL/SQLite), wykorzystując system mapowania obiektowo-relacyjnego (ORM) Entity Framework Core. Bazy relacyjne zapewniają zgodność z zasadami ACID, co jest kluczowe z punktu widzenia integralności skomplikowanych agregatów w DDD (np. zapewnienie, że relacje między dołączonymi urządzeniami, grupami a ich politykami dostępu są trwałe zachowane bez ryzyka wystąpienia niespójności).

3. Analiza i projektowanie systemu

3.1. Wymagania systemowe

Proces projektowania asystenta AI wymagał w pierwszej kolejności zdefiniowania jednoznacznych wymagań, które system musi spełniać, aby sprostać zidentyfikowanym problemom biznesowym i technicznym. Zebrano je i podzielono na wymagania funkcjonalne (określające, co system ma robić) oraz нефункционалне (określające, jak system ma działać pod kątem jakości, wydajności i ograniczeń technicznych).

3.1.1. Wymagania funkcjonalne

Do najważniejszych wymagań funkcjonalnych stawianych przed systemem należą:

- **Zarządzanie grupami urządzeń:** Użytkownik musi mieć możliwość utworzenia nowej grupy w systemie (otrzymując unikalny kod dostępu) oraz dołączenia do istniejącej grupy z poziomu dowolnego urządzenia klienckiego.
- **Zarządzanie wtyczkami (ang. *plugins*):** Aplikacja kliencka musi automatycznie wykrywać i łądować skompilowane wtyczki (pliki `.dll`) z określonego katalogu lokalnego. Użytkownik może z poziomu interfejsu klienta instalować, włączać oraz wyłączać poszczególne wtyczki.
- **Przetwarzanie zapytań w języku naturalnym:** System musi umożliwić użytkownikowi wprowadzanie poleceń tekstowych lub głosowych z poziomu dowolnego urządzenia należącego do grupy.
- **Agregacja możliwości urządzeń:** Serwer centralny musi posiadać funkcję agregowania metadanych wszystkich aktywnych wtyczek i ich funkcji dla całej grupy urządzeń, w celu przekazania ich jako kontekstu do modelu LLM.
- **Autoryzacja wykonania funkcji:** Przed rozesłaniem poleceń wykonawczych (RPC) do klientów, serwer powinien zaprezentować użytkownikowi listę zinterpretowanych intencji (funkcji do wywołania wraz z argumentami) w celu ich weryfikacji i zatwierdzenia.
- **Rozproszone wykonanie zadania:** Po akceptacji przez użytkownika, system musi przekazać żądania wywołania funkcji do odpowiednich urządzeń, zebrać wyniki i zaprezentować je użytkownikowi.

3.1.2. Wymagania нефункционалне

Z perspektywy architektury oraz środowiska wykonawczego, zdefiniowano następujące wymagania нефункционалне:

- **Rozszerzalność (ang. *Extensibility*):** System musi pozwalać na wdrażanie nowych funkcjonalności (obsługi nowych urządzeń lub akcji) bez konieczności rekonfiguracji, zatrzymywania lub modyfikacji kodu źródłowego serwera.
- **Komunikacja o niskich opóźnieniach:** Opóźnienia w dystrybucji poleceń z serwera do urządzeń klienckich nie mogą przekraczać wartości akceptowalnych dla interaktywnych systemów sterowania. Wymusza to użycie technologii działającej w czasie rzeczywistym, takiej jak WebSocket.

205 • **Agnostyczność wobec dostawców AI:** Mechanizm mapowania zapytań języka naturalnego na struk-
206 tury danych musi być niezależny od konkretnego dostawcy modelu sztucznej inteligencji, pozwalając
207 na łatwą zmianę silnika LLM.

208 • **Modularność logiki po stronie serwera:** Kod serwera musi być ustrukturyzowany w sposób umoż-
209 liwiający łatwy podział i odseparowanie poszczególnych domen wiedzy (zgodnie z architekturą mo-
210 dułową).

211 3.2. Modelowanie przypadków użycia systemu

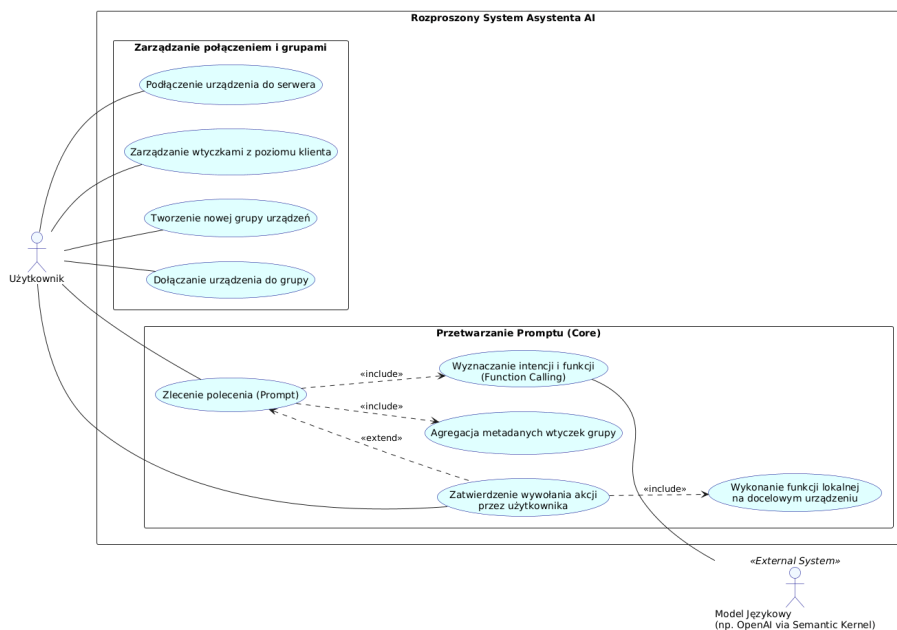
212 3.2.1. Identyfikacja aktorów i głównych procesów biznesowych

213 Podczas modelowania przypadków użycia (ang. *Use Cases*) zidentyfikowano głównego aktora systemu:
214 **Użytkownika końcowego**. Wchodzi on w interakcję z systemem za pośrednictwem aplikacji klienckiej
215 (np. komputera, telefonu). Do aktorów systemowych (pomocniczych) należą również **Zewnętrzny model**
216 **LLM**, z którym komunikuje się serwer centralny w celu parsowania intencji.

217 Główny proces biznesowy to „Zlecenie wykonania rozproszonego zadania”, w którym użytkownik wy-
218 powiada komendę (np. „Wycisz mój telefon i uruchom przeglądarkę na komputerze”). System, na podstawie
219 połączonych metadanych urządzeń w grupie, interpretuje to jako dwa niezależne wywołania funkcji, żąda
220 potwierdzenia od użytkownika i koordynuje równoległe ich wykonanie.

221 3.2.2. Diagramy przypadków użycia

222 W celu wizualizacji kluczowych interakcji użytkownika z aplikacją kliencką oraz serwerem, opraco-
223 wano diagram przypadków użycia w standardzie UML. Przypadki te obejmują między innymi tworzenie
224 grupy, synchronizację stanu połączenia, zarządzanie lokalnymi wtyczkami oraz cykl życia samego promptu.



Rys. 3.1. Diagram przypadków użycia dla systemu asystenta AI

3.3. Architektura systemu

Zgodnie z założeniami zdefiniowanymi w analizie teoretycznej (zob. Rozdział 2), system operuje w oparciu o architekturę klient-serwer. Logika biznesowa została wyraźnie rozdzielona pomiędzy centralny punkt koordynacyjny a agentów wykonawczych wyposażonych w dynamiczne moduły.

3.3.1. Architektura Modular Monolith i podział na konteksty (ang. *Bounded Contexts*)

Aplikacja serwerowa implementuje wzorec modułowego monolitu. Główny proces został logicznie podzielony na hermetyczne obszary (ang. *Bounded Contexts*), co gwarantuje wysoką spójność wewnętrzną. Wyróżniono następujące moduły:

- **Connection:** Zarządza tożsamością urządzeń i żywotnością fizycznych połączeń opartych na protokole WebSockets.
- **Groups:** Implementuje logikę biznesową powiązaną z tworzeniem przestrzeni współdzielonych i generowaniem kodów dostępu.
- **Plugins:** Agreguje nadsyłane z klientów struktury danych opisujące interfejsy wtyczek.

- **Prompts:** Odpowiada za bezpośrednią integrację z warstwą sztucznej inteligencji, budując kontekst i mapując wyniki przy użyciu warstwy pośredniczącej.

- **Tasks:** Śledzi przepływ zdalnych wywołań od momentu zlecenia aż do odebrania wyniku (lub błędu) z urządzenia końcowego.

3.3.2. Struktura warstwowa modułów (DDD i CQRS)

Wewnętrzna struktura każdego z wymienionych modułów opiera się na czterech warstwach: aplikacyjnej (*Application*), domenowej (*Domain*), infrastruktury (*Infrastructure*) oraz integracyjnej (*Integration*). Modele i reguły biznesowe izolowane są w warstwie domeny, która pozbawiona jest zewnętrznych zależności technicznych. Dostęp do operacji aplikacyjnych realizowany jest z wykorzystaniem wzorca CQRS (ang. *Command Query Responsibility Segregation*), oddzielając komendy mutujące stan (np. *CreateGroupCommand*) od zapytań (np. *GetGroupDetailsQuery*). Wymiana informacji między modułami, aby zachować ich niezależność, przebiega w pełni asynchronicznie poprzez publikację zdarzeń (ang. *Integration Events*).

3.3.3. Architektura wtyczek na urządzeniach klienckich

Aplikacja kliencka pełni rolę środowiska wykonawczego. Główny proces programu przy starcie skanuje zawartość katalogu `plugins/`. Po wykryciu skompilowanej biblioteki `.dll`, aplikacja za pomocą mechanizmu *Reflection* ładuje moduł i ekstrahuje metadane (nazwy i opisy klas, metod oraz ich parametrów wejściowych). Serwer nigdy nie dysponuje binarnym kodem wtyczki, zapewniając tym samym wysoki stopień separacji bezpieczeństwa i elastyczność dystrybucji oprogramowania po stronie użytkownika.

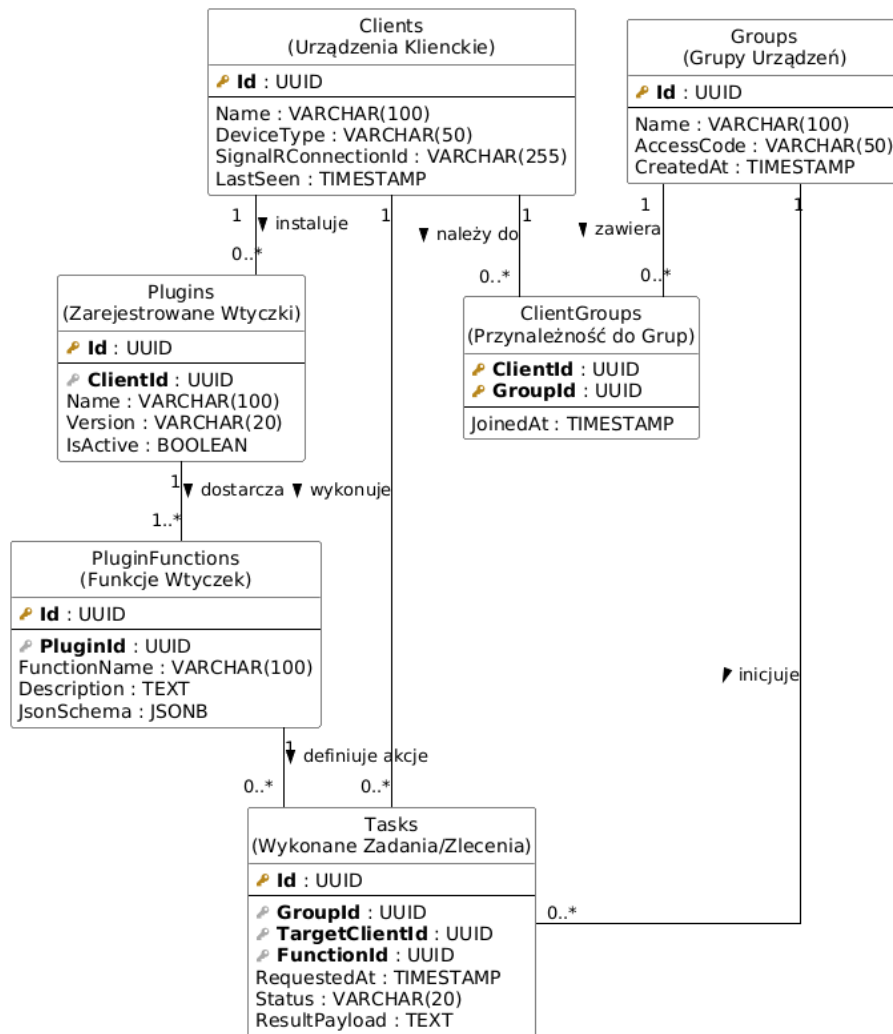
3.4. Projekt bazy danych

3.4.1. Modelowanie danych z perspektywy domenowej

Zgodnie z zasadami DDD, model bazy danych oparty jest na persystencji zamkniętych agregatów. Oznacza to, że baza danych operuje raczej na spójnych stanach całych obiektów biznesowych (np. *Grupa* zawierająca powiązane informacje autoryzacyjne) niż na zdenormalizowanych tabelach silnie związanych kluczami obcymi ponad granicami poszczególnych modułów. Za dostarczanie abstrakcji bazy danych odpowiada mechanizm ORM (Entity Framework Core), który dokonuje translacji modeli encji dziedzicznych na struktury relacyjne.

3.4.2. Diagram związków encji (ERD)

Diagram związków encji (ang. *Entity-Relationship Diagram*) przedstawia relacyjny schemat bazodanowy obejmujący najważniejsze z punktu widzenia działania systemu tabele: przechowujące stan logowania urządzeń klienckich (Klienci), definicje przestrzeni współdzielonych (Grupy) wraz z tablicą asocjacyjną powiązań (Urządzenia w Grupach), a także tabele archiwizujące zarejestrowane definicje akcji (Funkcje Wtyczek) oraz statusy ich wykonania (Zadania).



Rys. 3.2. Diagram relacyjny bazy danych (ERD) systemu

271 **4. Implementacja systemu**

272 **4.1. Implementacja logiki serwera i komunikacji między modułami**

273 **4.1.1. Struktura i odpowiedzialność głównych modułów biznesowych**

274 **4.1.2. Propagacja zdarzeń integracyjnych i wzorzec *Event-Driven***

275 **4.2. Implementacja po stronie klienta**

276 **4.2.1. Mechanizm dynamicznego ładowania zestawów przy użyciu refleksji**

277 **4.2.2. Rejestracja metadanych wtyczek na serwerze**

278 **4.3. Implementacja przepływu przetwarzania zapytań użytkow-** 279 **nika**

280 **4.3.1. Agregacja metadanych i budowa kontekstu zapytania**

281 **4.3.2. Wykorzystanie Semantic Kernel do parsowania poleceń**

282 **4.4. Obsługa dwukierunkowej komunikacji przy użyciu SignalR**

5. Testowanie i walidacja

5.1. Założenia i środowisko testowe

5.2. Scenariusze testowe

5.2.1. Testowanie procesów tworzenia i łączenia urządzeń w grupy

5.2.2. Walidacja poprawności dynamicznego ładowania wtyczek

5.3. Weryfikacja działania mechanizmu *function calling*

5.4. Analiza wydajności rozgłaszania wiadomości za pomocą SignalR

291 **6. Podsumowanie**

292 **6.1. Ocena stopnia realizacji celu pracy i wymagań**

293 **6.2. Napotkane problemy i sposoby ich rozwiązania**

294 **6.3. Perspektywy dalszego rozwoju aplikacji**

296

Spis rysunków

297

3.1	Diagram przypadków użycia dla systemu asystenta AI	14
-----	--	----

298

3.2	Diagram relacyjny bazy danych (ERD) systemu	16
-----	---	----

299 **Spis tabel**

Streszczenie pracy

Projekt i implementacja rozproszonego asystenta AI z modułową architekturą wtyczek

W dobie dynamicznego rozwoju sztucznej inteligencji, systemy klasy asystentów głosowych i tekstowych (takie jak Google Assistant, Amazon Alexa czy Apple Siri) stały się integralną częścią codziennego życia. Mimo swojej popularności, komercyjne rozwiązania tego typu charakteryzują się istotnymi ograniczeniami wynikającymi z zamkniętości ich ekosystemów. Integracja nowych urządzeń lub dodanie niestandardowych funkcjonalności wymaga z reguły korzystania z dedykowanych pakietów SDK, przechodzenia przez rygorystyczne procesy certyfikacji oraz całkowitego uzależnienia się od infrastruktury chmurowej zewnętrznego dostawcy. Skutkuje to brakiem interoperacyjności pomiędzy urządzeniami różnych producentów, a także rodzi obawy dotyczące prywatności. Co więcej, istniejące systemy nie oferują natywnego mechanizmu grupowania różnorodnych urządzeń we wspólną, spójną przestrzeń sterowaną za pomocą pojedynczego polecenia.

Odpowiedzią na wyżej wymienione problemy jest niniejsza praca inżynierska, której głównym celem było zaprojektowanie oraz implementacja rozproszonego, modularnego i w pełni otwartego systemu asystenta AI. Skonstruowane rozwiązanie umożliwia intuicyjne sterowanie heterogenicznym zbiorem urządzeń końcowych (m.in. komputerami osobistymi, smartfonami czy smartwatchami) przy użyciu poleceń wydawanych w języku naturalnym. Fundamentalnym założeniem projektowym była otwartość architektury, pozwalająca na łatwe i dynamiczne rozszerzanie możliwości systemu o obsługę nowych urządzeń oraz dziedzin bez konieczności ingerencji w jego kod źródłowy.

Architektura opracowanego systemu opiera się na modelu klient-serwer. Serwer centralny, zaimplementowany w języku C#, został zaprojektowany zgodnie z paradygmatem modular monolith z rygorystycznym zastosowaniem metodyki Domain-Driven Design (DDD) oraz wzorca CQRS (Command Query Responsibility Segregation). Taki podział architektoniczny pozwolił na wyraźne wyznaczenie granic kontekstów (bounded contexts) dla poszczególnych modułów systemu (m.in. zarządzanie połączeniami, grupami, wtyczkami i przetwarzaniem zapytań). Ograniczyło to narzut operacyjny charakterystyczny dla rozproszonych architektur opartych na mikroservisach, przy jednoczesnym zachowaniu wysokiej spójności i podatności na dalszy rozwój. Za dwukierunkową, niskoopóźnieniową komunikację pomiędzy klientami a serwerem odpowiada biblioteka SignalR, zapewniająca niezawodny transport danych w czasie rzeczywistym.

Kluczowym elementem warunkującym rozszerzalność systemu jest innowacyjna architektura pluginów po stronie klienckiej. Funkcjonalności specyficzne dla danego urządzenia są dostarczane w postaci dynamicznie ładowanych bibliotek .dll. W momencie startu, aplikacja kliencka ładuje dostępne moduły do pamięci, po czym rejestruje ich metadane w serwerze centralnym, pozostawiając sam kod wykonawczy wyłącznie na urządzeniu użytkownika. Dzięki temu serwer pełni rolę bezstanowego orkiestratora, nieposiadającego wiedzy o implementacji poszczególnych operacji.

Centralnym bytem logicznym zaprojektowanego rozwiązania są grupy urządzeń. Mechanizm ten pozwala użytkownikowi na łączenie wielu fizycznych maszyn na podstawie wygenerowanego kodu dostępu, tworząc ujednoliconą przestrzeń operacyjną. Gdy użytkownik wydaje polecenie (prompt), serwer agreguje metadane wszystkich aktywnych pluginów we wskazanej grupie i konstruuje rozszerzony kontekst dla

340 dużego modelu językowego (LLM). Integracja z modelami AI została zrealizowana przy użyciu biblioteki
341 Microsoft Semantic Kernel, która stanowi warstwę abstrakcji, uniezależniając logikę biznesową od konkret-
342 nego dostawcy LLM. Dzięki zastosowaniu mechanizmu function calling, model poddaje analizie intencję
343 użytkownika i zwraca optymalny zestaw funkcji do wykonania. Po autoryzacji operacji przez użytkownika,
344 serwer dystrybuuje polecenia do docelowych klientów, gdzie są one wykonywane natywnie.

345 Opracowany system stanowi otwarty ekosystem klasy smart assistant. Zaprojektowana architektura sku-
346 tecznie rozwiązuje problem interoperacyjności i rozszerzalności, pozwalając na współpracę wielu zróżni-
347 cowanych urządzeń. Wykorzystanie nowoczesnych wzorców inżynierii oprogramowania w połączeniu z
348 potęgą dużych modeli językowych umożliwiło stworzenie narzędzia, które stanowi skalowalną alter-
349 natywę dla rozwiązań komercyjnych, zapewniając użytkownikowi niezależność i kontrolę nad własnym
350 środowiskiem cyfrowym.

Abstract

In the era of rapid development of artificial intelligence, voice and text assistant systems (such as Google Assistant, Amazon Alexa, or Apple Siri) have become an integral part of everyday life. Despite their popularity, commercial solutions of this type are characterized by significant limitations resulting from the closed nature of their ecosystems. Integrating new devices or adding custom functionalities typically requires utilizing dedicated SDKs, undergoing rigorous certification processes, and becoming entirely dependent on a third-party provider's cloud infrastructure. This results in a lack of interoperability between devices from different manufacturers and raises privacy concerns. Furthermore, existing systems do not offer a native mechanism for grouping diverse devices into a common, cohesive space controlled by a single command.

The response to the aforementioned problems is this engineering thesis, the main objective of which was to design and implement a distributed, modular, and fully open AI assistant system. The developed solution enables intuitive control over a heterogeneous set of end devices (including personal computers, smartphones, and smartwatches) using natural language commands. A fundamental design premise was the openness of the architecture, allowing for easy and dynamic extension of the system's capabilities to support new devices and domains without modifying its source code.

The architecture of the developed system is based on the client-server model. The central server, implemented in C#, was designed according to the modular monolith paradigm with a rigorous application of the Domain-Driven Design (DDD) methodology and the CQRS (Command Query Responsibility Segregation) pattern. Such an architectural division allowed for the clear delineation of bounded contexts for individual system modules (e.g., managing connections, groups, plugins, and prompt processing). This approach mitigated the operational overhead characteristic of distributed microservice architectures while maintaining high cohesion and extensibility for future development. Bidirectional, low-latency communication between clients and the server is handled by the SignalR library, ensuring reliable, real-time data transport.

A crucial element ensuring the system's extensibility is the innovative plugin architecture on the client side. Device-specific functionalities are delivered as dynamically loaded .dll libraries. Upon startup, the client application loads available modules into memory and registers their metadata with the central server, leaving the execution code exclusively on the user's device. Consequently, the server acts as a stateless orchestrator, lacking knowledge of the implementation details of individual operations.

The central logical entity of the designed solution is device groups. This mechanism allows the user to connect multiple physical machines based on a generated access code, creating a unified operational space. When a user issues a command (prompt), the server aggregates the metadata of all active plugins within the specified group and constructs an extended context for the Large Language Model (LLM). Integration with AI models was achieved using the Microsoft Semantic Kernel library, which serves as an abstraction layer, decoupling the business logic from any specific LLM provider. By utilizing the function calling mechanism, the model analyzes the user's intent and returns the optimal set of functions to execute. Following the user's authorization of the operation, the server distributes commands to the target clients, where they are executed natively.

389 The developed system constitutes an open smart assistant ecosystem. The designed architecture effec-
390 tively addresses the problems of interoperability and extensibility, enabling the collaboration of numerous
391 diverse devices. Utilizing modern software engineering patterns in conjunction with the power of Large
392 Language Models has enabled the creation of a tool that serves as a scalable alternative to commercial
393 solutions, ensuring the user's independence and full control over their digital environment.

394

395

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

396

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

397

..... Piotr Prezydent

398

Imię (imiona) i nazwisko studenta

399

400

Wydział Nauk Ścisłych i Technicznych

401

402

..... Informatyka

403

Nazwa kierunku

404

405

..... 131497

406

Numer albumu

407

1. Oświadczam, że moja praca dyplomowa pt.: Projekt i implementacja rozproszonego asystenta AI z modułarną architekturą wtyczek

408

409

1) została przygotowana przeze mnie samodzielnie*,

410

2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,

411

412

3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,

413

414

4) nie była podstawą nadania dyplomu uczelni wyższej ani mnie, ani innej osobie.

415

2. Jednocześnie wyrażam zgodę/ nie wyrażam zgody** na udostępnienie mojej pracy dyplomowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

416

417

418

(miejscowość, data)

419

(czytelny podpis studenta)

420

* Uwzględniając merytoryczny wkład promotora pracy

421

* – niepotrzebne skreślić

422

423

Załącznik nr 3 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

424

OŚWIADCZENIE STUDENTA O ZGODNOŚCI WERSJI PAPIEROWEJ I ELEKTRONICZNEJ PRACY

425

426

..... Piotr Prezydent

427

Imię (imiona) i nazwisko studenta

428

429

Wydział Nauk Ścisłych i Technicznych

430

431

..... Informatyka

432

Nazwa kierunku

433

434

..... 131497

435

Numer albumu

436

Oświadczam, że treść pracy zamieszczonej przeze mnie w Systemie Wirtualna Uczelnia i zatwierdzonej

437

przez promotora, jest identyczna z wersją drukowaną oraz zawartą na nośniku elektronicznym.

438

439

(miejscowość, data)

(czytelny podpis studenta)